Discord Productivity Bot

Project design and implementation plan

Scope	**Single-user Discord bot for a private server; no multi-user or public SaaS concerns.**
Stack	TypeScript, Discord bot framework, Todoist API, Google Calendar API, SQLite, scheduled jobs.
Primary outcome	A reliable personal assistant for tasks, events, reminders, daily digests, and quick capture.
Future extension	Optional LLM layer for natural-language planning and summarization after deterministic flows are stable.

Prepared as a build-ready planning document for implementation.

1. Context and project goals

This project is a personal Discord bot that lives inside a private server used only by you and the bot. The server acts as a structured workspace rather than a generic chat surface. The bot should help you review today's obligations, inspect upcoming work, create and manage tasks, create and manage calendar events, and send reminders into purpose-specific channels.

The system is intentionally single-user. That changes the architecture in useful ways: there is no need for multi-tenant account linking, public onboarding, per-user permissions, or generalized administration. The design can therefore be opinionated around your workflow and your preferred channel structure.

The first implementation should be fully deterministic. No LLM is required to make the bot useful. The bot should already work well through slash commands, button-based interactions, scheduled digests, and direct Todoist and Google Calendar integrations. Natural-language handling can be deferred or limited to deterministic parsing rules until a later phase.

Design principles

- Keep Todoist as the source of truth for tasks.
- Keep Google Calendar as the source of truth for events.
- Store only local configuration, sync metadata, reminder jobs, and conversation context needed for bot behavior.
- Prefer explicit slash commands over hidden automation in the MVP.
- Structure the private server by workflow: capture, review, reminders, and planning.
- Build every core feature without any LLM dependency so that later AI features are additive rather than foundational.

2. Recommended technology choices

Use TypeScript end to end. This keeps the bot runtime, HTTP integration code, scheduling code, and local tooling in one language, reduces context-switching, and lets you benefit from strong typing around API contracts and domain models.

Layer	Choice	Why	Notes
Runtime	Node.js + TypeScript	One language for bot, integrations, and tooling.	Use strict TypeScript settings.
Discord	discord.js	Mature TypeScript-first bot ecosystem.	Use slash commands, buttons, selects, and modals.
Task backend	Todoist REST API	Richer task model than Google Tasks.	Projects, labels, priorities, due dates.
Calendar backend	Google Calendar API	Full event CRUD and mature ecosystem.	Use OAuth or service-compatible flow for your

			own account.
Database	SQLite	Simple, local, adequate for a single-user system.	Use Prisma or Drizzle if you want migrations and types.
Jobs	node-cron or Agenda/Bree	Daily digests and reminder polling.	Start simple; polling is fine for MVP.
HTTP server	Fastify or Express	OAuth callback endpoints and health checks.	Can be omitted until account linking is needed.
Config	.env + typed config module	Small deployment footprint.	Validate config on startup.

3. Target feature set

Core feature goals

Feature	Goal
Daily review	Post a daily summary of today's tasks and today's calendar events into #today.
Upcoming review	Show upcoming tasks and events for the next 7 days and broader monthly views.
Task management	Create, edit, complete, reopen, and delete Todoist tasks from Discord.
Calendar management	Create, edit, and delete Google Calendar events from Discord.
Reminders	Post deadline and event reminders into #reminders.
Channel specialization	Use separate channels for capture, summaries, reminders, planning, and audit/logging if desired.
Natural language support	Later phase. In MVP, support explicit commands and optionally limited deterministic parsing in #inbox.

Non-goals for the first implementation

- Multi-user support and per-user account linking.
- Cross-server installation or bot distribution.
- Heavy autonomous planning behavior.
- Complex synchronization from multiple task providers at once.
- Full natural-language understanding or agentic behavior.

4. Private server channel design

The private server should separate passive information from active interaction. That keeps routine output readable and makes the server itself part of the product design.

Channel	Purpose	Posting pattern	MVP priority
#inbox	Quick capture and command entry.	Manual commands and optional message capture.	High
#today	Morning digest and on-demand daily view.	Scheduled each morning; /today replies can also be mirrored here.	High
#reminders	Deadline reminders and event alerts.	Bot posts triggered reminders.	High
#planning	Weekly review and longer-range planning views.	Manual commands such as /week and /month.	Medium
#log	Audit trail for actions taken by the bot.	Optional append-only success/error summaries.	Low

Channel usage guidance

- #inbox is the primary interaction surface. Even if slash commands are globally available, this should feel like the default place to interact with the bot.
- #today is optimized for scanning, so avoid posting chatty command confirmations there.
- #reminders should contain only actionable reminders, not full daily summaries.
- #planning can host longer command outputs such as weekly and monthly views that would clutter #today.

5. MVP architecture

The MVP architecture should stay narrow and deterministic. The bot layer receives commands and button interactions, the application layer validates and translates them into domain actions, the integration layer talks to Todoist and Google Calendar, and SQLite stores local configuration and metadata.

Architecture layers

Layer	Responsibilities	Typical modules
Presentation	Slash commands, buttons, modals, embeds, channel posting, scheduled message rendering.	discord command handlers, embed builders
Application	Validate inputs, resolve defaults, orchestrate operations, map results to response DTOs.	task service, calendar service, reminder service
Integration	Call external APIs and normalize responses.	todoist client, google calendar client
Persistence	Store config, IDs, sync metadata, reminders, and logs.	SQLite schema, repository layer
Scheduling	Run digests and reminder scans on a schedule.	cron jobs, job runner, startup scheduler

Suggested folder structure

```
src/
  bot/            # discord client, command registration, interaction handlers
  app/            # task/calendar/reminder services and use cases
  integrations/
    todoist/
    google-calendar/
  db/             # schema, repositories, migrations
  jobs/           # daily digest and reminder polling
  config/         # environment parsing and runtime settings
  domain/         # types, DTOs, validation, enums
  utils/
```

Core runtime flow

- Discord interaction -> command handler
- command handler -> application service
- application service -> external integration(s) + repository
- application service -> response DTO
- handler -> embed/message response in the appropriate channel

6. Domain model and local database schema

Because Todoist and Google Calendar remain the system of record, the local database does not need to duplicate all external data. It stores only what the bot needs locally for configuration, reminder delivery, command defaults, and action traceability.

Table	Purpose	Important columns	Notes
app_config	Singleton config for the one-user bot.	timezone, defaultCalendarId, digestChannelId, remindersChannelId, inboxChannelId	One row is enough.
oauth_tokens	Store provider auth state.	provider, accessToken, refreshToken, expiryUtc, scopeBlob	Can be skipped if using manually provisioned tokens.
todoist_task_map	Optional local cache or alias mapping.	todoistTaskId, lastSeenContent, lastSeenDueUtc	Useful for faster lookups and stable reminder handling.
calendar_event_map	Optional local cache for event metadata.	googleEventId, calendarId, startUtc, endUtc, summary	Useful for reminders and edits.
reminder_jobs	Track reminders the bot should emit.	jobId, sourceType, sourceId, remindAtUtc, channelId, deliveredAtUtc, status	Core for #reminders.
action_log	Audit log of performed actions.	actionType, sourceCommand, payloadJson, resultJson, createdAtUtc	Helpful for debugging.

Minimal TypeScript entity sketch

```
type Provider = 'todoist' | 'google-calendar';

interface AppConfig {
  timezone: string;
  inboxChannelId: string;
  todayChannelId: string;
  remindersChannelId: string;
  planningChannelId?: string;
  defaultCalendarId: string;
}

interface ReminderJob {
  id: string;
  sourceType: 'task' | 'event';
  sourceId: string;
  remindAtUtc: string;
  channelId: string;
```

```
  deliveredAtUtc?: string;
  status: 'pending' | 'delivered' | 'failed' | 'cancelled';
}
```

7. Command set

The MVP should focus on a compact command surface with predictable outputs. Group commands by resource and by review horizon.

Command	Intent	Primary output	MVP
/today	Show today's tasks and today's events.	Embed in #today or ephemeral reply.	Yes
/week	Show next 7 days of tasks and events.	Embed or paginated output.	Yes
/month	Show broader monthly view.	Condensed grouped output.	Yes
/task add	Create Todoist task.	Confirmation + link or task summary.	Yes
/task edit	Change title, due date, priority, project, or labels.	Confirmation with before/after.	Yes
/task done	Complete task.	Confirmation and optional log entry.	Yes
/task delete	Delete task.	Confirmation with undo guidance if applicable.	Yes
/event add	Create Google Calendar event.	Confirmation with event summary.	Yes
/event edit	Edit event fields.	Confirmation with before/after.	Yes
/event delete	Delete event.	Confirmation.	Yes
/sync	Manual health/sync check across providers.	Status summary.	Optional
/help	Show available commands and channel conventions.	Reference output.	Optional

Command option guidelines

- Prefer explicit options instead of overloading free text. Example: /task add title:<text> due:<date> priority:<1-4> project:<name>.
- Use Discord modals for edits when multiple fields may change together.
- Use buttons for safe, low-friction follow-ups such as complete, reopen, edit due date, or open in Todoist.
- Return compact embeds with consistent sections: tasks due today, overdue tasks, and events today.

8. End-to-end command flows

/today

- User runs /today in #today or #inbox.
- Discord handler loads app_config and timezone.
- Task service fetches Todoist tasks due today plus overdue tasks you still care to display.
- Calendar service fetches Google Calendar events for the current day.
- Application layer merges and sorts the output into a single response model.
- Embed renderer formats sections: overdue, due today, events today, quick actions.
- Bot posts the result in channel or replies to the command invocation.

/week

- User runs /week.
- Bot requests tasks and events for the next seven days.
- Application layer groups items by day using the configured timezone.
- Renderer builds one section per day with concise bullets or fields.
- Bot replies in #planning or #today depending on command usage.

/task add

- User invokes /task add with title and optional due date, project, labels, and priority.
- Command validator checks required fields and normalizes date input.
- Todoist client sends create request.
- Result is written to action_log and optional reminder_jobs are generated if the task has a due time or reminder policy.
- Bot replies with a compact task summary and next actions.

/task edit

- User invokes /task edit task:<selector> ...changes...
- Bot resolves the task selector either through a recent cache or a fresh Todoist lookup.
- Validator compares requested changes with existing fields.
- Todoist client updates the task.
- Bot refreshes local reminder jobs if due data changed.
- Bot responds with before/after summary.

/task done

- User invokes /task done task:<selector>.
- Bot resolves the task ID and sends complete request to Todoist.
- Bot cancels related pending reminder_jobs.
- Action is logged and confirmation is sent.

/event add

- User invokes /event add with title, start, end, and optional location/description.
- Validator checks start < end and applies default calendar ID.
- Google Calendar client creates the event.
- Bot schedules reminders according to local policy.
- Bot responds with an event summary including date, time, and calendar.

/event edit

- User invokes /event edit event:<selector> ...changes...
- Bot resolves event ID, loads current event data, validates the patch, and sends update request.
- Reminder jobs are updated if relevant fields changed.
- Bot replies with before/after summary.

/event delete

- User invokes /event delete event:<selector>.
- Bot resolves event ID and deletes it from Google Calendar.
- Pending reminders are cancelled and the action is logged.
- Bot confirms deletion.

9. Scheduled jobs

Job	Schedule	Responsibility	MVP
Morning digest	Daily at chosen local time	Post summary to #today with tasks and events for the day.	Yes
Reminder scan	Every 1-5 minutes	Find due reminders and post them to #reminders.	Yes
Cleanup	Daily	Mark stale delivered reminders and prune old cache entries.	Optional
Manual sync health	On demand	Check provider	Optional

		connectivity and report token/permission issues.	

For the MVP, polling is acceptable. You do not need webhook-based synchronization on day one. A frequent reminder scan plus fresh API reads for command-driven views will keep the system simple and reliable.

10. Natural-language support without an LLM

You mentioned natural-language parsing in #inbox. In the deterministic phase, this should be treated as convenience, not as the primary interaction model. Start with slash commands. Then add a narrow parser only for a few common patterns if desired.

- Good deterministic pattern: 'task buy groceries tomorrow 18:00'.
- Good deterministic pattern: 'event dentist 2026-04-03 10:00 11:00'.
- Avoid trying to interpret ambiguous language like 'later this week' in the MVP.
- If Todoist's own natural-language date parsing is available through the chosen create flow, use it carefully but still echo back the resolved result to confirm what was interpreted.

11. Phased implementation plan

Phase	Goal	Main deliverables	Exit criteria
Phase 0	Foundation	Repo setup, TypeScript config, env loading, Discord bot login, SQLite schema bootstrap.	Bot starts locally and responds to a /ping or /help command.
Phase 1	Read-only views	/today, /week, /month, channel posting, Todoist read integration, Google Calendar read integration.	You can inspect tasks and events from Discord reliably.
Phase 2	Task mutations	/task add, edit, done, delete; action logging; optional task cache.	All task CRUD-like flows work from Discord.
Phase 3	Event mutations	/event add, edit, delete; reminders tied to events.	Calendar event management works from Discord.
Phase 4	Reminders and digests	Morning digest, reminder scan, reminder delivery, cleanup jobs.	Routine automated output works without manual commands.
Phase 5	Inbox convenience	Limited deterministic text capture in #inbox, modals, buttons, quality-of-life polish.	The bot feels smooth enough for daily use.
Phase 6	Future AI extension	Optional natural-language planning, summarization, and prioritization.	Only after deterministic flows are trusted.

12. Concrete build steps

1. Create the repository and choose package manager, linting, formatter, and tsconfig baseline.
2. Stand up a minimal discord.js client and register a few slash commands.
3. Add SQLite and the first migration for app_config, action_log, and reminder_jobs.
4. Implement Todoist read client and a /today task-only view.
5. Implement Google Calendar read client and merge it into /today.
6. Build /week and /month grouped views.
7. Implement task mutation commands and confirmation embeds.
8. Implement event mutation commands and confirmation embeds.
9. Add scheduled morning digest and reminder scan jobs.
10. Polish channel behavior, logging, and failure handling.

13. Risks and decisions to make early

Decision	Why it matters	Recommended default
ORM vs raw SQL	Affects migration workflow and type modeling.	Use Prisma or Drizzle if you want fast iteration; raw SQL is also fine for a small bot.
OAuth flow vs pre-generated tokens	Determines whether you need an HTTP callback service immediately.	For a personal bot, start with the simplest secure auth path you can maintain.
Reminder policy	Determines how much local scheduling logic you need.	Start with one reminder channel and a small set of fixed reminder offsets.
Task selector UX	Editing commands need a stable way to pick the right item.	Support recent-item selectors and explicit IDs only if needed.

14. Initial recommendation

Build the project in this order: foundation, read-only views, task mutation, event mutation, then reminders and inbox convenience. Keep the architecture modular, but do not over-engineer it. The single-user scope is your advantage. Use it to make the bot opinionated, reliable, and pleasant inside your private server.

The first milestone worth shipping is simple: /today, /week, /task add, /task done, and a morning digest to #today. Once that loop feels good, the rest of the system will be much easier to extend.